

quantum gates in this article. Recall that there are a large number of quantum gates, and it is neither practical nor necessary to discuss all of them at once. Instead, we will gradually introduce more and more gates as the series progresses. We shall begin by familiarising ourselves with the various single-qubit quantum gates before moving on to multi-qubit gates, which form the building blocks of more sophisticated quantum algorithms.

In this article, we will expand our knowledge of quantum gates by studying the Pauli gates, named after Wolfgang Pauli, the brilliant Austrian theoretical physicist who was awarded the 1945 Nobel Prize in Physics. Once again, we encounter a Nobel laureate whose work has had a profound influence on quantum computing—a recurring theme in the history of this fascinating field.

There are three Pauli gates, namely the Pauli-X, Pauli-Y, and Pauli-Z gates. All three are single-qubit quantum gates and play a fundamental role in quantum computing. Let us begin by examining the Pauli-X gate and see how it is implemented in Qiskit. Consider the program `'qiskit4.py'` (with line numbers included for clarity), shown below.

```
1. from qiskit import QuantumCircuit
2. from qiskit_aer import AerSimulator
3. qc = QuantumCircuit(1, 1)
4. qc.x(0)
5. qc.measure(0, 0)
6.
7. sim = AerSimulator()
8. result = sim.run(qc, shots=1000).
   result()
9. counts = result.get_counts()
10. print(counts)
```

The program shown above is almost identical to `'qiskit3.py'` discussed earlier. The only difference is in Line 4, where the Hadamard gate has been replaced by the Pauli-X gate. Line 4 applies the Pauli-X gate to qubit 0. Since a

newly created qubit is initialised in the state $|0\rangle$, the action of the Pauli-X gate transforms it to the state $|1\rangle$. In this sense, the Pauli-X gate plays a role analogous to that of the classical NOT gate, which changes 0 to 1 and 1 to 0.

Before examining the output of the program `'qiskit4.py'`, let us modify it slightly by inserting another line of code `'qc.x(0)'` (identical to Line 4), immediately after Line 4. The resulting program will be referred to as `'mod_qiskit4.py'`.

```
(qiskit_env) # python qiskit4.py
{'1': 1000}
(qiskit_env) # python mod_qiskit4.py
{'0': 1000}
```

Figure 3: Effect of applying the Pauli-X gate once and twice

Figure 3 shows the outputs obtained from the programs `'qiskit4.py'` and `'mod_qiskit4.py'`. In `'qiskit4.py'`, a single Pauli-X gate is applied to a qubit initially in the state $|0\rangle$. Consequently, the qubit is transformed to the state $|1\rangle$, and all 1000 measurements produce the outcome 1. In `'mod_qiskit4.py'`, the Pauli-X gate is applied twice in succession. The first application changes the state from $|0\rangle$ to $|1\rangle$, while the second application changes it back from $|1\rangle$ to $|0\rangle$. As a result, all 1000 measurements yield the outcome 0. Thus, applying the Pauli-X gate twice restores the original state of the qubit. This demonstrates an important property of the Pauli-X gate: it is reversible and acts as its own inverse. In other words, two successive applications of the Pauli-X gate are equivalent to performing no operation at all on the qubit. Notice the contrast with the Hadamard gate: here, the outcome is completely deterministic, with no randomness involved.

In the previous article in the series, we discussed the Bloch sphere and used it to visualise the effect of the Hadamard gate on a qubit. Let us now revisit the Bloch sphere to understand the action of the Pauli-X gate. Recall that the state $|0\rangle$ is represented by an arrow pointing towards the north pole of the Bloch

sphere, while the state $|1\rangle$ corresponds to an arrow pointing towards the south pole. Figure 4 shows the effect of applying the Pauli-X gate to a qubit initially in the state $|0\rangle$. It can be seen that the state vector is rotated from the north pole to the south pole, thereby transforming the state $|0\rangle$ into $|1\rangle$. If the Pauli-X gate is applied once again, the state vector returns to the north pole, demonstrating that the Pauli-X gate is reversible and acts as its own inverse.

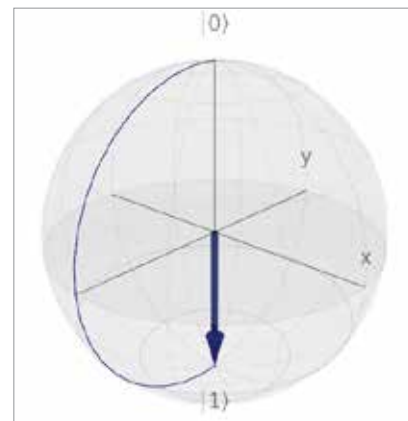


Figure 4: Effect of the Pauli-X gate on $|0\rangle$

Let us now try to understand the mathematics behind the operation of the Pauli-X gate. Recall that, in vector form, the states $|0\rangle$ and $|1\rangle$ are represented by $[1\ 0]^T$ and $[0\ 1]^T$, respectively. Notice the superscript T, which indicates that these are column vectors. In the previous article, we saw how the Hadamard gate is represented by a 2×2 matrix. Similarly, the Pauli-X gate is also represented by a 2×2 matrix. Figure 5 shows the matrix representation of the Pauli-X gate. The figure also illustrates how the Pauli-X matrix acts on the column vector corresponding to $|0\rangle$ and transforms it into the column vector corresponding to $|1\rangle$. It would be a worthwhile exercise to perform the same matrix multiplication on the column vector representing $|1\rangle$ and verify that the result is the column vector corresponding to $|0\rangle$. This confirms that the action of the Pauli-X gate on the computational basis states is analogous to that of the classical NOT gate, which